

Derivation of Categorical Cross-Entropy Loss: A Step-by-Step Probabilistic Foundation

Faculty of Artificial Intelligence — Menoufia University

Natural Language Processing / Data Science Fundamentals | 2025–2026

Abstract

The categorical cross-entropy loss is the most widely used objective function in multi-class classification tasks, yet it is often introduced as a formula without derivation. This paper provides a complete, rigorous, step-by-step derivation of the cross-entropy loss from first principles, starting from probability theory and Maximum Likelihood Estimation (MLE). We derive the **softmax function** from the requirement that raw logit scores must form a valid probability distribution. We then show how the **categorical cross-entropy loss** follows directly and inevitably from the negative log-likelihood of the observed labels under the categorical distribution. The elegant gradient $\partial L / \partial z_k = \hat{y}_k - y_k$ is derived analytically. The relationship between cross-entropy and **Kullback–Leibler (KL) divergence** is established, revealing that training with cross-entropy is equivalent to minimizing the information-theoretic distance between the predicted and true distributions. Common misconceptions — including why MSE is unsuitable for classification — are addressed. Extensions including label smoothing are discussed.

1. Probability Foundations

1.1 Discrete Random Variables and Probability Distributions

A **random variable** is a function that maps outcomes of a random experiment to real numbers. In classification, the output of interest is a discrete random variable Y that takes one of C possible class labels (e.g., $C = 10$ for the 10 digit classes in MNIST). A valid **probability distribution** over these classes must satisfy two conditions:

$$0 \leq p(Y = c) \leq 1 \quad \text{for all } c, \quad \text{and} \quad \sum_{c=1}^C p(Y = c) = 1$$

Non-negativity ensures probabilities are meaningful; the normalization condition ensures they form a complete description of the random variable. Any function mapping classes to outputs that satisfies both conditions is a valid categorical distribution.

1.2 Likelihood and Log-Likelihood

When we have a dataset of N independent samples $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$, the **likelihood** of the entire dataset given model parameters θ is the joint probability of all observations:

$$L(\theta) = \prod_{i=1}^N p(y^{(i)} \mid x^{(i)}; \theta)$$

Because each factor is a number between 0 and 1, their product over N samples becomes numerically vanishingly small. Taking the natural logarithm converts the product to a sum (since $\log(a \cdot b) = \log a + \log b$) while preserving the ordering of values (since log is monotonically increasing):

$$\log L(\theta) = \sum_{i=1}^N \log p(y^{(i)} \mid x^{(i)}; \theta)$$

Maximizing the log-likelihood is mathematically equivalent to maximizing the likelihood itself. This is the objective of **Maximum Likelihood Estimation (MLE)** — choosing the model parameters θ that make the observed data most probable under the model.

2. Classification as Probabilistic Estimation

In modern neural networks, the final layer does not output a single deterministic class prediction. Instead, it outputs a **probability distribution over all C classes**, estimating the conditional probability $p(Y = c \mid x; \theta)$ for each class c given input x and current parameters θ .

The last linear layer of a neural network produces **logits** $z = [z_1, z_2, \dots, z_C]$ — raw, unbounded real-valued scores for each class. These logits do not satisfy the requirements of a probability distribution: they can be negative, greater than 1, and they do not necessarily sum to 1. We need a transformation that converts them into valid probabilities.

3. The Softmax Function

3.1 Derivation and Properties

The **softmax function** achieves the required transformation. For a logit vector $z \in \mathbb{R}^C$, the softmax probability for class i is:

$$\hat{y}_i = \text{softmax}(z)_i = \exp(z_i) / \sum_{j=1}^C \exp(z_j)$$

The softmax satisfies both probability conditions by construction:

- **Non-negativity:** $\exp(z_i) > 0$ for any real z_i , so $\hat{y}_i > 0$ always.

- **Normalization:** Dividing by $\sum \exp(z_j)$ ensures $\sum_i \hat{y}_i = 1$ exactly.

3.2 Numerical Stability: The Log-Sum-Exp Trick

For large logit values (e.g., $z_i = 1000$), $\exp(z_i)$ overflows to infinity. In practice, a constant $M = \max(z)$ is subtracted from all logits before exponentiation:

$$\text{softmax}(z)_i = \exp(z_i - M) / \sum_j \exp(z_j - M)$$

This is mathematically identical to the original definition (the M terms cancel in numerator and denominator) but prevents numerical overflow. The largest logit now maps to $\exp(0) = 1$, and all others map to values in $(0, 1]$.

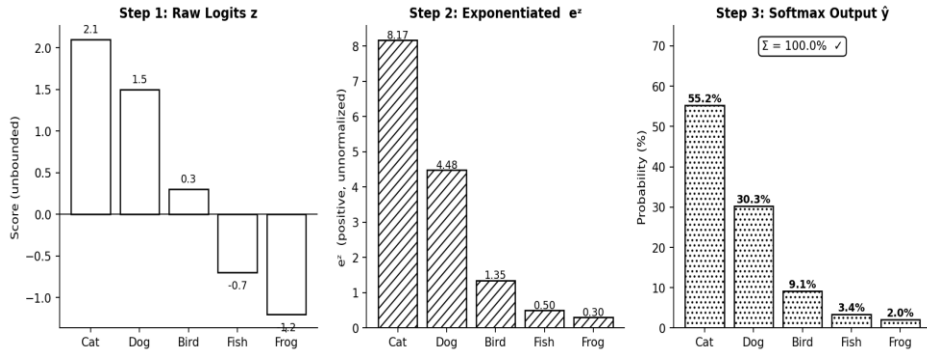


Figure 1. Three-step softmax transformation on five-class logits. Step 1: Raw logits z (unbounded, can be negative). Step 2: Exponentiated values $\exp(z)$ — all positive but unnormalized. Step 3: Softmax output $\hat{y}$ — valid probabilities summing to exactly 100%.

4. One-Hot Encoding and Ground Truth Representation

The true label for each sample is a single class index c . To compare it directly with the model's probability vector $\hat{y} \in \mathbb{R}^C$, we represent it as a **one-hot vector**:

$$y^{(i)} = [0, 0, \dots, 1, \dots, 0]^T \in \{0, 1\}^C$$

where the entry at position c equals 1 (the true class) and all other entries equal 0. This representation satisfies $y_{-c}^{(i)} \in \{0, 1\}$ and $\sum_c y_{-c}^{(i)} = 1$, making it a valid (degenerate) probability distribution — one that places all probability mass on a single class.

One-hot encoding places the ground truth and the model output in the same vector space, enabling **direct comparison between the two distributions** using information-theoretic measures such as cross-entropy.

5. Maximum Likelihood Estimation — The Core Derivation

5.1 Likelihood of a Single Sample

We model each training example as drawn from a **categorical distribution** parameterized by the model's softmax output $\hat{y}^{(i)}$. The probability of observing the true label $y^{(i)}$ for sample i is, using one-hot encoding:

$$p(y^{(i)} \mid x^{(i)}; \theta) = \prod_{\{c=1\}^C} (\hat{y}_{-c}^{(i)})^{\{y_{-c}^{(i)}\}}$$

Because $y^{(i)}$ is one-hot, only one term in this product has an exponent of 1 (the true class $c^{(i)}$); all others have exponent 0 and equal 1. The product therefore collapses to:

$$p(y^{(i)} \mid x^{(i)}; \theta) = \hat{y}_{c(i)}^{(i)}$$

This is simply the softmax probability assigned to the correct class — an elegantly simple expression.

5.2 Likelihood of the Full Dataset

Assuming the N training samples are independent and identically distributed (i.i.d.), the joint likelihood factorizes:

$$L(\theta) = \prod_{i=1}^N \prod_{c=1}^C (\hat{y}_{c(i)}^{(i)})^{y_{c(i)}^{(i)}}$$

5.3 Log-Likelihood

Taking the natural logarithm converts the nested products to sums:

$$\begin{aligned} \log L(\theta) &= \sum_{i=1}^N \log \left(\prod_{c=1}^C (\hat{y}_{c(i)}^{(i)})^{y_{c(i)}^{(i)}} \right) \\ &= \sum_{i=1}^N \sum_{c=1}^C y_{c(i)}^{(i)} \cdot \log(\hat{y}_{c(i)}^{(i)}) \end{aligned}$$

Because $y^{(i)}$ is one-hot, the inner sum has only one non-zero term (where $y_{c(i)}^{(i)} = 1$), simplifying to $\log(\hat{y}_{c(i)}^{(i)})$ per sample. The double summation is retained for its **vectorization convenience** and because it generalizes immediately to soft labels (label smoothing, mixup, etc.).

MLE Derivation Pipeline: From Dataset to Cross-Entropy Loss

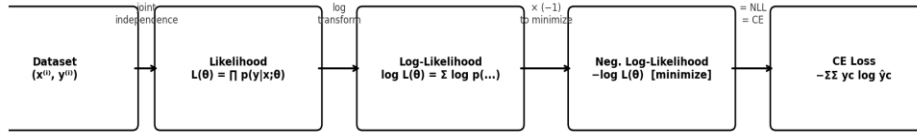


Figure 2. MLE derivation pipeline: starting from the dataset, the joint independence assumption gives the likelihood. The log transform converts products to sums. Negating produces a minimizable loss. The final expression is categorical cross-entropy.

6. From Log-Likelihood to Cross-Entropy Loss

Deep learning frameworks minimize objective functions. Since we want to **maximize** the log-likelihood, we multiply by -1 to obtain the **Negative Log-Likelihood (NLL)**, which we minimize:

$$L(\theta) = -\log L(\theta) = -\sum_{i=1}^N \sum_{c=1}^C y_{c(i)}^{(i)} \cdot \log(\hat{y}_{c(i)}^{(i)})$$

This is precisely the **categorical cross-entropy loss**. It is not a heuristic formula — it is the **direct negative log-likelihood** of the observed labels under the categorical distribution produced by softmax. By minimizing $L(\theta)$, we maximize the probability that the model assigns to the observed training labels, which is the MLE objective for multi-class classification.

For a single training example, since only one $y_{c(i)} = 1$, the loss simplifies further to:

$$L^{(i)} = -\log(\hat{y}_{c(i)}^{(i)})$$

where $c^{(i)}$ is the true class. This has an intuitive interpretation: if the model predicts probability 0.9 for the correct class, the loss is $-\log(0.9) \approx 0.105$ (low). If it predicts 0.01, the loss is

$-\log(0.01) \approx 4.6$ (very high). The loss grows rapidly as the model becomes more confidently wrong.

7. Cross-Entropy: Information-Theoretic Interpretation

The expression derived above is also the definition of **cross-entropy** between the true distribution y (one-hot) and the predicted distribution $\hat{y}$:

$$H(y, \hat{y}) = -\sum_{c=1}^C y_c \cdot \log(\hat{y}_c)$$

Cross-entropy has a precise information-theoretic meaning: it measures the **average number of bits required to encode events from the true distribution** y using a coding scheme optimized for the predicted distribution $\hat{y}$. If the model's predictions perfectly match the true labels ($\hat{y} = y$), the cross-entropy equals the Shannon entropy $H(y)$ — the theoretical minimum. Any mismatch between $\hat{y}$ and y incurs additional bits of encoding cost, which is the loss the model must minimize.

8. Why the Softmax + Cross-Entropy Gradient Is Elegant

8.1 Derivation of $\partial L / \partial z_k$

One of the most remarkable properties of combining softmax with cross-entropy is the simplicity of the resulting gradient with respect to the logits z . Starting from the per-sample loss $L = -\log(\hat{y}_{\{c(i)\}})$, we apply the chain rule:

$$\partial L / \partial z_k = \sum_j (\partial L / \partial \hat{y}_j) \cdot (\partial \hat{y}_j / \partial z_k)$$

The softmax Jacobian has two cases: $\partial \hat{y}_i / \partial z_k = \hat{y}_i(1 - \hat{y}_i)$ if $i = k$, and $\partial \hat{y}_i / \partial z_k = -\hat{y}_i \cdot \hat{y}_k$ if $i \neq k$.

After substitution and algebraic simplification, the result is beautifully compact:

$$\partial L / \partial z_k = \hat{y}_k - y_k$$

The gradient is simply the **prediction minus the ground truth for each class**. This remarkable simplification occurs because the softmax and cross-entropy are mathematically paired: the exponential in softmax cancels with the logarithm in cross-entropy, leaving a linear residual.

8.2 Desirable Properties of This Gradient

- **Numerical stability:** The gradient is bounded and well-behaved — no large exponents or logarithms appear at inference time.
- **Computational efficiency:** A single subtraction per logit. No expensive operations during backpropagation beyond the softmax forward pass.
- **Intuitive direction:** For the true class, $\hat{y}_c - y_c = \hat{y}_c - 1 \leq 0$ — the gradient pushes the logit up (increases probability). For all other classes, $\hat{y}_k - 0 = \hat{y}_k \geq 0$ — pushes their logits down.
- **Strong signal for confident errors:** If the model predicts $\hat{y}_c = 0.01$ for the true class, the gradient is -0.99 , a large corrective signal. MSE would give only $-0.02^2 \cdot 2 \approx 0.04$ — far weaker.

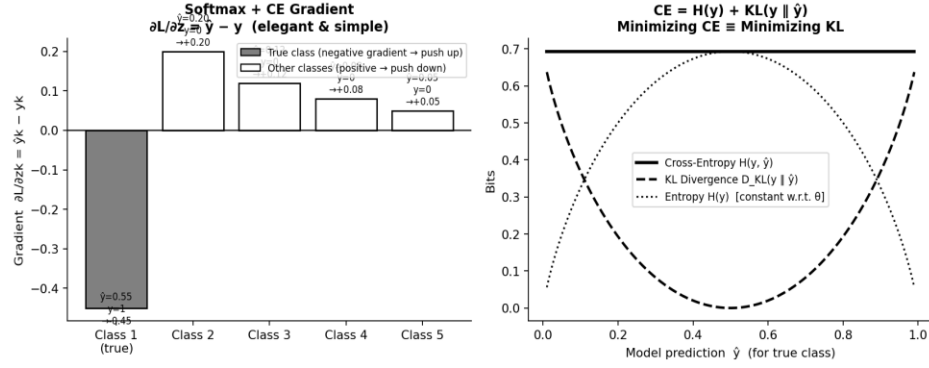


Figure 3. Left: Gradient $\partial L / \partial z_k = \hat{y}_k - y_k$ for a 5-class example. The true class (Class 1) receives a negative gradient (push logit up); all other classes receive positive gradients (push logits down). Right: Cross-entropy $H(y, \hat{y})$ decomposes into fixed entropy $H(y)$ plus KL divergence $D_{KL}(y || \hat{y})$. Since $H(y)$ is constant w.r.t. model parameters, minimizing CE is equivalent to minimizing KL.

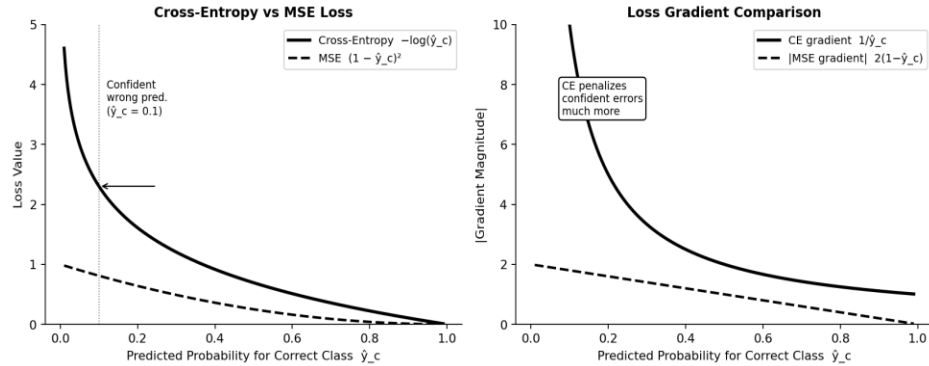


Figure 4. Left: Cross-entropy vs MSE loss as a function of predicted probability for the correct class. CE grows to infinity as $\hat{y}_c \rightarrow 0$ (extreme penalty for confident wrong predictions); MSE is bounded. Right: CE gradient magnitude grows rapidly for confident errors ($1/\hat{y}_c \rightarrow \infty$), ensuring strong corrective signals; MSE gradient vanishes for small $\hat{y}_c$.

9. Why the Double Summation Appears

The full batch loss $\mathcal{L}(\theta) = -\sum \sum y_{c^{(i)}} \log(\hat{y}_{c^{(i)}})$ contains a double summation: the **outer sum** over N training samples, and the **inner sum** over C classes. Although the one-hot encoding makes all but one term in the inner sum zero for hard labels, the double sum is retained for two important reasons:

- **Vectorization:** Modern deep learning frameworks compute the inner product $\mathbf{y} \cdot \log(\hat{\mathbf{y}})$ as a single matrix operation over the entire batch, regardless of the number of non-zero terms.
- **Generality:** The same formula works for **soft labels** — when labels are probability distributions rather than one-hot vectors (e.g., from label smoothing, knowledge distillation, or multi-label mixup). In these cases, multiple terms in the inner sum are non-zero.

10. Comparison: Cross-Entropy vs Mean Squared Error

Criterion	Cross-Entropy Loss	Mean Squared Error (MSE)
Formula	$-\sum y_c \log(\hat{y}_c)$	$\sum (y_c - \hat{y}_c)^2 / C$
Statistical basis	Negative log-likelihood of categorical distribution	MLE under Gaussian noise assumption
Gradient w.r.t. logits	$\hat{y}_k - y_k$ (simple, stable)	Complex; involves softmax Jacobian
Penalty for confident wrong prediction	Extremely high ($\rightarrow \infty$ as $\hat{y}_c \rightarrow 0$)	Modest (bounded by 1)
Gradient saturation	None — gradient stays large for wrong predictions	Yes — gradient $\rightarrow 0$ near 0 or 1
Suitable for classification	Yes — designed for probability outputs	No — assumes continuous regression targets
Numerical stability	Excellent with log-sum-exp trick	Good but less expressive

Why MSE fails for classification: MSE assumes the residuals follow a Gaussian distribution, which is appropriate for regression but not for discrete class labels. More critically, MSE applies only **weak penalty for confident wrong predictions**: a model predicting $\hat{y}_{\text{correct}} = 0.01$ (confidently wrong) incurs $\text{MSE} \approx (0.01-1)^2 = 0.98$, while cross-entropy incurs $-\log(0.01) \approx 4.6$. The cross-entropy's logarithmic penalty forces the model to sharply distinguish correct from incorrect predictions.

11. Relation to Kullback–Leibler Divergence

Cross-entropy can be decomposed using Shannon entropy $H(y)$ and **KL divergence** $D_{\text{KL}}(y \parallel \hat{y})$:

$$H(y, \hat{y}) = H(y) + D_{\text{KL}}(y \parallel \hat{y})$$

where $H(y) = -\sum_c y_c \log(y_c)$ is the Shannon entropy of the true label distribution, and

$$D_{\text{KL}}(y \parallel \hat{y}) = \sum_c y_c \cdot \log(y_c / \hat{y}_c)$$

measures the **"extra surprise"** incurred by using the model distribution $\hat{y}$ to encode events from the true distribution y . Since the labels y are fixed (determined by the dataset), $H(y)$ is a constant with respect to the model parameters θ . Therefore:

$$\min_{\theta} H(y, \hat{y}) \iff \min_{\theta} D_{\text{KL}}(y \parallel \hat{y})$$

Training a neural network with categorical cross-entropy is **exactly equivalent to minimizing the KL divergence** between the empirical data distribution and the model's predicted distribution. This connects the loss function to the foundational principle of making the model's beliefs as close as possible to reality in the information-theoretic sense.

12. Common Misconceptions

- **Misconception 1:** "Cross-entropy is an arbitrary choice." **Reality:** Cross-entropy is the statistically principled objective for categorical data — the unique loss that follows from MLE under a categorical distribution. Any other loss (including MSE) involves a different (incorrect) distributional assumption.
- **Misconception 2:** "MSE can be used for classification." **Reality:** MSE assumes Gaussian noise on continuous outputs. For probability outputs in $[0, 1]$, MSE produces weak gradients for confident errors (gradient $\rightarrow 0$ at prediction boundaries) and fails to push probabilities sharply toward 0 or 1.
- **Misconception 3:** "The loss is a formula from a library." **Reality:** Every term in categorical cross-entropy traces back to the likelihood of observed labels under the model's probabilistic output. The gradient $\hat{y} - y$ is not an implementation choice; it follows directly from the chain rule applied to the softmax–logit relationship.
- **Misconception 4:** "Softmax probabilities can be trusted as calibrated confidence." **Reality:** Modern deep networks trained with cross-entropy tend to be overconfident. Post-hoc calibration methods (temperature scaling, Platt scaling) are required for reliable uncertainty estimation.

13. Extensions and Modern Applications

13.1 Label Smoothing

Instead of hard one-hot targets $y_c \in \{0, 1\}$, label smoothing uses soft targets:

$$y_c^{\text{smooth}} = (1 - \varepsilon) \cdot y_c + \varepsilon / C$$

where ε is a small constant (typically 0.1). This prevents the model from becoming overconfident by reserving a small probability mass ε/C for each non-correct class. Label smoothing is standard in modern Transformers (BERT, ViT) and improves both calibration and generalization.

13.2 Focal Loss

Introduced by Lin et al. (2017) for object detection with extreme class imbalance, focal loss down-weights well-classified examples:

$$FL(\hat{y}_c) = -(1 - \hat{y}_c)^{\gamma} \cdot \log(\hat{y}_c) \quad (\gamma \geq 0)$$

When $\gamma = 0$, focal loss equals standard CE. For $\gamma > 0$, the term $(1 - \hat{y}_c)^{\gamma}$ reduces the contribution of easy examples (where $\hat{y}_c$ is already high), focusing training on hard, misclassified examples.

13.3 Temperature Scaling and Calibration

The softmax temperature parameter T controls distribution sharpness:

$$\hat{y}_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

At $T = 1$, standard softmax. $T > 1$ softens the distribution (more uniform, lower confidence); $T < 1$ sharpens it. Temperature scaling after training with a held-out calibration set is the most effective method for producing well-calibrated neural network probabilities.

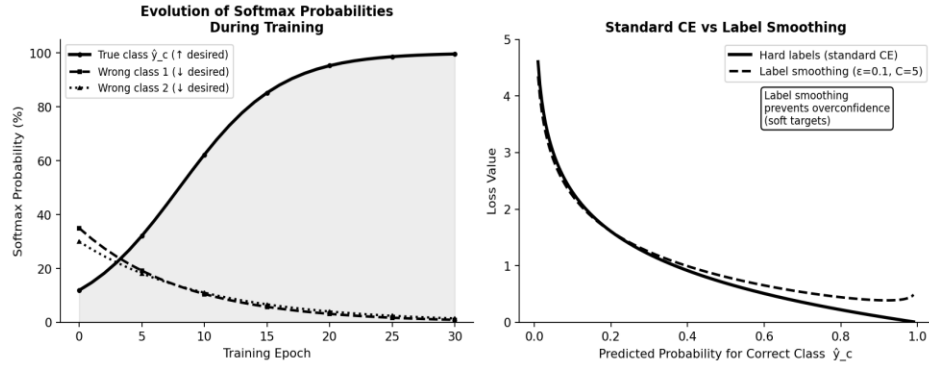


Figure 5. Left: Evolution of softmax probabilities during training — the true class probability (solid) rises toward 1 while incorrect class probabilities (dashed, dotted) are pushed toward 0. Right: Label smoothing ($\epsilon=0.1$) vs standard CE — smoothing prevents the loss from reaching zero and reduces overconfidence by maintaining non-zero probability mass on incorrect classes.

14. Summary of Key Formulas

Concept	Formula	Intuition
Softmax	$\hat{y}_i = \exp(z_i) / \sum_j \exp(z_j)$	Converts logits to valid probability distribution
Likelihood (single sample)	$p(y x;\theta) = \prod \hat{y}_c^{y_c}$	Probability of observing the true label
Log-likelihood	$\log L(\theta) = \sum \sum y_c \log(\hat{y}_c)$	Sum of log probs; numerically stable
CE Loss (batch)	$L(\theta) = -\sum \sum y_c \log(\hat{y}_c)$	Negative log-likelihood; minimize this
CE Loss (single sample)	$L(i) = -\log(\hat{y}_{c(i)})$	$-\log$ of predicted probability for true class
Gradient w.r.t. logits	$\partial L / \partial z_k = \hat{y}_k - y_k$	Prediction minus ground truth per class
CE and KL divergence	$H(y, \hat{y}) = H(y) + D_{KL}(y \parallel \hat{y})$	CE = entropy + divergence; min CE $\equiv$ min KL

15. Conclusion

This paper has derived the categorical cross-entropy loss from first principles, revealing it to be the **inevitable consequence of treating classification as maximum likelihood estimation** of a categorical distribution. The key steps are:

- **Softmax** converts arbitrary logits into a valid probability distribution via exponentiation and normalization.
- **One-hot encoding** places the ground truth in the same vector space as the model output.

- **MLE** provides the principled objective: maximize the probability the model assigns to observed labels.
- **Negating the log-likelihood** gives a minimizable loss — categorical cross-entropy.
- **The gradient $\partial L / \partial z_k = \hat{y}_k - y_k$** follows directly from the chain rule and is numerically elegant.
- **$CE = H(y) + KL(y \parallel \hat{y})$** : minimizing cross-entropy is equivalent to minimizing KL divergence between true and predicted distributions.

Understanding this derivation removes the "black box" feel of the cross-entropy loss and equips practitioners to **debug training failures**, **design custom losses** (focal loss, label smoothing), **interpret gradient signals**, and **reason about model calibration**. Every architectural choice — from softmax to one-hot encoding — has a principled probabilistic motivation that traces back to the fundamental goal of statistical inference.

References

- [1] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27, 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 6: Deep Feedforward Networks. <https://www.deeplearningbook.org>
- [3] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 4: Linear Models for Classification. <https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
- [4] Kullback, S., & Leibler, R. A. (1951). On Information and Sufficiency. *Annals of Mathematical Statistics*, 22(1), 79–86. <https://doi.org/10.1214/aoms/1177729694>
- [5] Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. <https://arxiv.org/abs/1708.02002>
- [6] Szegedy, C., et al. (2016). Rethinking the Inception Architecture for Computer Vision (label smoothing introduced). *Proceedings of the IEEE CVPR*. <https://arxiv.org/abs/1512.00567>
- [7] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1706.04599>
- [8] Mehta, S. (2021). Deriving Categorical Cross-Entropy and Softmax. *Technical Blog*. <https://shivammehta25.github.io/posts/deriving-categorical-cross-entropy-and-softmax/>
- [9] Stanford CS231n Course Notes: Neural Networks Part 2 — Setting up the Data and the Loss. *Stanford University*. <https://cs231n.github.io/neural-networks-2/>
- [10] MacKay, D. J. C. (2003). *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press. Chapter 2: Probability, Entropy, and Inference. <https://www.inference.org.uk/itprnn/book.pdf>